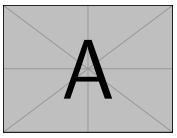
MetaCode

Research Roadmap, Implementation Strategy and MVP Product Plan

From Computational Meta-Optics Research to an Affordable Photonics Design Platform



Forward-looking Technical and Product Document

Companion to the MetaCode / MetaOpticsAI repository

October 6, 2026

About This Document

This document is a *forward-looking* companion to the MetaCode repository. It is not a user manual and it is not a research paper. It is a sequenced, cost-aware roadmap for turning an exploratory computational meta-optics codebase into (a) a reproducible research tool, (b) an internal design platform, and finally (c) a narrow, affordable MVP for photonics researchers.

The document deliberately treats any large-language-model (LLM) layer as a late, optional stage. The core scientific engine must stand on its own. Any provider mentioned (Groq, local open-source models, hosted APIs) is a candidate for a later, optional orchestration layer and not a prerequisite for the roadmap below.

Tag legend. [NOW] do this now; [NEXT] immediately after the current stage; [LATER] further down the roadmap; [MVP] specifically in scope for the minimum viable product; [FUTURE] post-MVP; [OPTIONAL] nice to have, do not block on it.

Contents

About This Document **2**

1 Vision **10**
1.1 What MetaCode Should Become . . . 10
1.2 Three Audiences . . . 10
1.3 Guiding Principles . . . 10
1.4 Scope and Non-Scope . . . 11
1.5 The Shape of the Ladder . . . 11

2 Current State of the Repository **13**
2.1 What Exists . . . 13
2.2 What Is Partial . . . 14
2.3 What Is Missing Entirely . . . 14
2.4 Starting Point for the Roadmap . . . 14

3 Target Architecture **16**
3.1 One-Picture Architecture . . . 16
3.2 Boundaries . . . 16
3.3 Design Invariants . . . 17
3.4 Where the Current Code Fits . . . 17

4 Research Roadmap **19**
4.1 Shape of the Research Programme . . . 19
4.2 Stage-to-Paper Mapping . . . 19
4.3 Open-Access Priority . . . 19
4.4 Research Hygiene . . . 21

5 Stage 1 – Scientific Foundations **22**
5.1 Objective . . . 22
5.2 Why Necessary . . . 22
5.3 Required Theory . . . 22
5.4 Implementation Tasks . . . 23
5.5 Expected Inputs and Outputs . . . 23
5.6 Validation Criteria . . . 23
5.7 Open-Source Tools . . . 23

5.8 Hardware Requirements . . . 23
5.9 What NOT to Build Yet . . . 24
5.10 Deliverable and Exit Criterion . . . 24

6 Stage 2 – Unit-Cell / Meta-Atom Engine 25
6.1 Objective . . . 25
6.2 Why Necessary . . . 25
6.3 Required Theory . . . 25
6.4 Algorithms . . . 25
6.5 Implementation Tasks . . . 26
6.6 Inputs / Outputs . . . 26
6.7 Validation Criteria . . . 26
6.8 Papers . . . 26
6.9 Open-Source Tools . . . 27
6.10 Hardware Requirements . . . 27
6.11 What NOT to Build Yet . . . 27
6.12 Deliverable and Exit Criterion . . . 27

7 Stage 3 – Meta-Atom Database and Phase Library 28
7.1 Objective . . . 28
7.2 Why Necessary . . . 28
7.3 Required Theory . . . 28
7.4 Database Schema (SQLite) . . . 28
7.5 Implementation Tasks . . . 29
7.6 Inputs / Outputs . . . 29
7.7 Validation Criteria . . . 29
7.8 Papers . . . 30
7.9 Open-Source Tools . . . 30
7.10 Hardware Requirements . . . 30
7.11 What NOT to Build Yet . . . 30
7.12 Deliverable and Exit Criterion . . . 30

8 Stage 4 – Metalens Layout and Full-Aperture Simulation 31
8.1 Objective . . . 31
8.2 Why Necessary . . . 31
8.3 Required Theory . . . 31
8.4 Design Pipeline . . . 31
8.5 Implementation Tasks . . . 32
8.6 Inputs / Outputs . . . 32
8.7 Validation Criteria . . . 32
8.8 Papers . . . 32
8.9 Open-Source Tools . . . 33
8.10 Hardware Requirements . . . 33

8.11 What NOT to Build Yet . . . 33
8.12 Deliverable and Exit Criterion . . . 33

9 Stage 5 – Electromagnetic Validation Backend 34
9.1 Objective . . . 34
9.2 Why Necessary . . . 34
9.3 Required Theory . . . 34
9.4 Backend Interface . . . 34
9.5 Implementation Tasks . . . 35
9.6 Inputs / Outputs . . . 35
9.7 Validation Criteria . . . 35
9.8 Papers . . . 35
9.9 Open-Source Tools . . . 35
9.10 Hardware Requirements . . . 35
9.11 What NOT to Build Yet . . . 36
9.12 Deliverable and Exit Criterion . . . 36

10 Stage 6 – Optimization and Inverse Design 37
10.1 Objective . . . 37
10.2 Why Necessary . . . 37
10.3 Required Theory . . . 37
10.4 Three Layers of Optimisation . . . 37
10.5 Implementation Tasks . . . 38
10.6 Validation Criteria . . . 38
10.7 Papers . . . 38
10.8 Open-Source Tools . . . 38
10.9 Hardware Requirements . . . 39
10.10 What NOT to Build Yet . . . 39
10.11 Deliverable and Exit Criterion . . . 39

11 Stage 7 – Automated Design Pipeline 40
11.1 Objective . . . 40
11.2 Why Necessary . . . 40
11.3 Pipeline . . . 40
11.4 Implementation Tasks . . . 40
11.5 Observability . . . 42
11.6 Validation Criteria . . . 42
11.7 Deliverable and Exit Criterion . . . 42

12 Stage 8 – Agent and Tool Orchestration 43
12.1 Objective . . . 43
12.2 Why Necessary . . . 43
12.3 Implementation Tasks . . . 43
12.4 Deliverable and Exit Criterion . . . 43

13 Stage 9 – Optional LLM Interface **45**
13.1 Objective . . . 45
13.2 Why This Stage Is Optional . . . 45
13.3 Design . . . 45
13.4 Providers . . . 46
13.5 Safety and Determinism . . . 46
13.6 Implementation Tasks . . . 46
13.7 Papers . . . 46
13.8 What NOT to Build Here . . . 47
13.9 Deliverable and Exit Criterion . . . 47

14 Stage 10 – Validation Programme **48**
14.1 Objective . . . 48
14.2 Why Necessary . . . 48
14.3 Validation Classes . . . 48
14.4 Implementation Tasks . . . 48
14.5 Deliverable and Exit Criterion . . . 49

15 Stage 11 – Productisation **50**
15.1 Objective . . . 50
15.2 Narrow Scope . . . 50
15.3 Who the First Users Are . . . 50
15.4 Product Surface . . . 50
15.5 Non-Features . . . 51
15.6 Deliverable and Exit Criterion . . . 51

16 MVP Architecture **52**
16.1 Target MVP . . . 52
16.2 One-Picture Architecture . . . 52
16.3 Folder Structure . . . 52
16.4 Flask Routes (Minimum) . . . 53
16.5 Database Schema (SQLite) . . . 53
16.6 Deployment Target . . . 54
16.7 Observability . . . 54
16.8 What the MVP Does Not Do . . . 54

17 MVP Implementation Plan **55**
17.1 Order of Work . . . 55
17.2 Minimum Feature Set . . . 55
17.3 Explicitly Not in the MVP . . . 55
17.4 Build Order Diagram . . . 56

18 Low-Cost Infrastructure **57**
18.1 Compute Budgets . . . 57

18.2 Precompute, Cache, Approximate . . . 57
18.3 Where GPUs Help – And Where They Don't . . . 58
18.4 Local Development on a Laptop . . . 58
18.5 Cloud Experimentation: Zero-Cost Options . . . 58
18.6 Production: What Lives on the Single VM . . . 58
18.7 Scaling Beyond the MVP . . . 59

19 Startup / Product Strategy 60
19.1 A Realistic Trajectory . . . 60
19.2 Who Pays First? . . . 60
19.3 Narrow Positioning . . . 61
19.4 Open Core vs. Commercial . . . 61
19.5 Early Customer Experiments . . . 61
19.6 Pricing Hypotheses . . . 62
19.7 Risks Specific to This Market . . . 62

20 Timeline, Milestones, and Risks 63
20.1 Milestone Table . . . 63
20.2 Dependencies Between Milestones . . . 63
20.3 Phase / Difficulty / Risk / Exit . . . 63
20.4 Risk Register . . . 63

21 Conclusion and Recommended Execution Order 66
21.1 Summary . . . 66
21.2 The One-Page Execution Order . . . 66
21.3 Final Words . . . 67

List of Figures

1.1 The MetaCode ladder. Blue stages are the scientific spine; green stages are automation; orange is the optional LLM; red is the product MVP. 12

3.1 Target architecture. Dashed edge = optional later-stage LLM layer. Nothing in the scientific core depends on the LLM. 16

11.1 The automated design pipeline. Every node emits an artifact handle; the report chains them into a reproducible trail. 41

13.1 Optional LLM interface. The language model only produces or edits a structured JSON specification; numeric claims in the final explanation must come from the tool outputs. 45

16.1 MVP architecture. Everything runs on one VM; the job manager is a thread pool, not Celery; storage is SQLite + HDF5 + `outputs/`. 52

17.1 MVP build order. 56

19.1 Suggested product trajectory. 60

20.1 Milestone dependency DAG. M14 (optional chat) is dashed; it is not a prerequisite for any other milestone. 64

List of Tables

2.1 Starting matrix for the roadmap. 15

3.1 Mapping of existing modules to the target architecture. 18

4.1 Stage → paper → algorithm → MetaCode implementation. 20

16.1 MVP Flask routes. 53

17.1 MVP feature set. 56

18.1 Compute budgets per development phase. 57

20.1 MetaCode milestone table. 63

20.2 Phases, difficulty, risk, exit criteria. 64

Chapter 1

Vision

1.1 What MetaCode Should Become

MetaCode's long-term ambition is modest but concrete: an *affordable*, *open*, *reproducible* meta-optics design platform that a graduate student with a laptop and a hobbyist cloud budget can use end-to-end – from parameter choice to a fabrication-ready layout. It should not try to replace Lumerical FDTD or Ansys HFSS. It should be the *first* tool a researcher reaches for when sketching a device, and a reliable spine that connects a chosen unit-cell geometry, a validated simulation, a focusing phase profile, and a GDSII mask.

This document plans the path. It plans it as a ladder, not as a project backlog: each stage builds on the previous, each stage has a crisp exit criterion, and the earliest stages produce value even if the later ones are never built.

1.2 Three Audiences

The document is written for three overlapping readers:

- **Researchers.** People designing meta-atoms and metalenses who want a free, open alternative to commercial CAD/EM tools and are willing to accept a smaller feature set in exchange.
- **Developers.** Contributors to the MetaCode codebase who want to know what to implement next, in what order, and why.
- **Founders and advisors.** Readers assessing whether there is a defensible, low-cost product opportunity near the research tool.

1.3 Guiding Principles

Six principles bound every design decision in the roadmap.

1. **Scientific correctness before everything else.** The design pipeline must produce physically meaningful numbers. Nothing downstream matters if the simulated phase is wrong.

2. **Reproducibility by construction.** Every result must be re-derivable from pinned inputs and a captured code revision.
3. **Lowest-cost path first.** Early stages run on a laptop. Occasional GPU access is a bonus, not a prerequisite.
4. **Open-source only in the critical path.** Any commercial dependency must be optional.
5. **AI/LLM is a sweetener, not a pillar.** The core engine works without a language model. Any model, hosted or local, is a late and optional convenience.
6. **Narrow MVP.** The first paying or sign-up product does *one* thing well.

1.4 Scope and Non-Scope

In scope for the roadmap. Dielectric-pillar metasurfaces (TiO_2, Si, GaN, SiN, Si_3N_4, and compatible materials), single-wavelength RCWA, Fresnel-zone metalenses, Fourier-optics PSF/MTF/Strehl/Zernike analysis, diameter-sweep libraries, GDSII export, optimisation of a scalar figure-of-merit, and a thin automation/orchestration layer that drives all of the above.

Out of scope for at least the first MVP. Full-wave 3-D vector simulation at full chip scale, multilayer process modelling, tolerance analysis for fabrication, multi-wavelength chromatic lens co-design, scalar-plus-vector hybrid synthesis, generative-ML meta-atom discovery, closed-loop measurement integration, and multi-tenant cloud deployment with SLAs. These are reasonable later ambitions; they are the wrong fights to pick now.

1.5 The Shape of the Ladder

Figure 1.1 is the whole roadmap in one picture. Every subsequent chapter expands one rung.

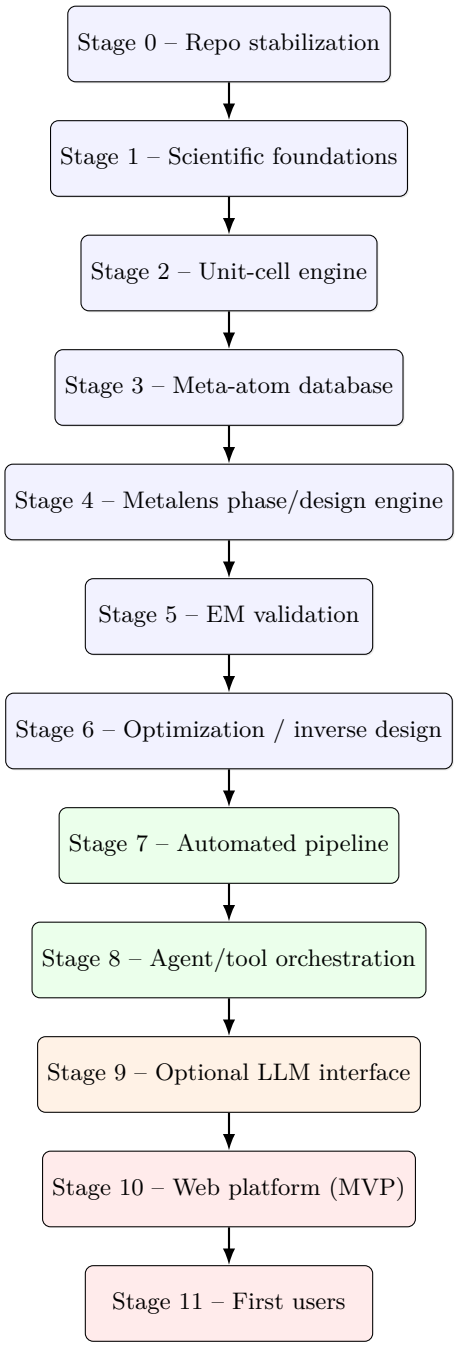


Figure 1.1: The MetaCode ladder. Blue stages are the scientific spine; green stages are automation; orange is the optional LLM; red is the product MVP.

Chapter 2

Current State of the Repository

This chapter is an honest snapshot of what exists today, so later chapters can plan from a known starting point. Where a module is only partially wired, it is marked as such.

2.1 What Exists

The MetaCode repository (`metaopticsai 0.2.0`) contains:

- A Pydantic-typed domain layer (`MetalensDesignParams`, `IncidenceSpec`, `Artifact`, five result dataclasses).
- A physics backend abstraction with three concrete backends: an always-available analytical heuristic, a TensorFlow-based RCWA path through the vendored `metabox3` solver, and an S4 stub that currently raises `NotImplementedError` on its three simulation methods.
- A phase-profile library (Fresnel zone lens, axicon, spiral phase plate) and a uniform-interval phase quantiser.
- Fourier-optics analysis: PSF, MTF (with diffraction-limited reference), Strehl ratio, Noll-indexed Zernike decomposition, encircled energy, far-field.
- Three optimiser strategies: a numpy-only heuristic that does not actually search, a TensorFlow gradient-based inverse-design path through metabox3's `LensAssembly`, and a surrogate skeleton that returns `status: not_implemented`.
- A GDSII exporter driven by `gdsfactory`.
- A broader scientific-tool catalogue (`src/metaopticsai/tools/`) with Pydantic input/output schemas and a `BaseTool`/`ToolRegistry` abstraction that composes about sixteen tools.
- A minimal, exploratory tool-calling scaffold under `llm/` and `mcp_server/` that, in principle, could orchestrate the six narrowest tools through a hosted language model. This path is *not* the project's scientific spine; this document treats it as an optional, later stage.
- A runnable `examples/demo.py` that exercises the narrow six-tool catalogue end-to-end.

2.2 What Is Partial

- Only three verified meta-atom templates ship.
- Only diameter sweeps are supported by both the metabox and analytical backends; other sweep axes raise `NotImplementedError`.
- The analytical backend uses a 4-material look-up table of fixed refractive indices; dispersion is only available via the CSV-based material facade and is not used in the agent path.
- The S4 backend is scaffolded but not implemented.
- The heuristic optimiser does not search; it reports a Strehl estimate from a pre-computed sweep.
- The surrogate optimiser is a skeleton.
- No stdio MCP server; no persistent artifact store; no test suite.

2.3 What Is Missing Entirely

- A reproducible FDTD validation path (Meep or openEMS).
- A reproducible RCWA cross-check (S4 or a second independent implementation).
- A persistent, queryable meta-atom database.
- A web interface.
- Documentation of design decisions (now partially addressed by the technical report companion to this document).
- A CI / regression suite tied to physics golden references.
- A dependency manifest free of UTF-16 encoding issues.
- A pinned LLM snapshot and a per-call cost/usage report (should the optional LLM stage ever go to production).

2.4 Starting Point for the Roadmap

Table 2.1 is the matrix the roadmap starts from: for each pillar, what exists, what is partial, what is missing.

Table 2.1: Starting matrix for the roadmap.

Pillar	**Exists**	**Missing / partial**
Domain model	Pydantic design/incidence/materials/results	Fabrication constraints; multi-λ sweeps
RCWA	metabox3 adapter + analytical heuristic	S4 backend; dispersion in metabox path; non-diameter sweeps
FDTD	Static reference library of five wavelengths	Meep / openEMS integration
Phase lib	FZL, axicon, SPP + quantiser	Chromatic / achromatic composite phase profiles
Analysis	PSF, MTF, Strehl, Zernike, EE, far-field	Vector field metrics; tolerance analysis
Optimization	Gradient path through metabox3	Working heuristic / surrogate; Optuna integration
Fabrication	GDSII writer (gdsfactory)	Design-rule checker; cross-tool export (CIF, DXF)
Tool layer	16 tools, Pydantic schemas, registry	Persistent store; transport (stdio MCP, HTTP)
LLM layer	Exploratory scaffold (narrow 6-tool loop)	Treated as optional, future stage
Infrastructure	`outputs/` path, env settings, logging	Database, web UI, deploy target

Chapter 3

Target Architecture

3.1 One-Picture Architecture

Figure 3.1 is the target end-state. The scientific engine sits in the middle; everything above it is interchangeable (CLI, web UI, optional LLM). Everything below it is swappable (RCWA backend, FDTD backend, surrogate model).

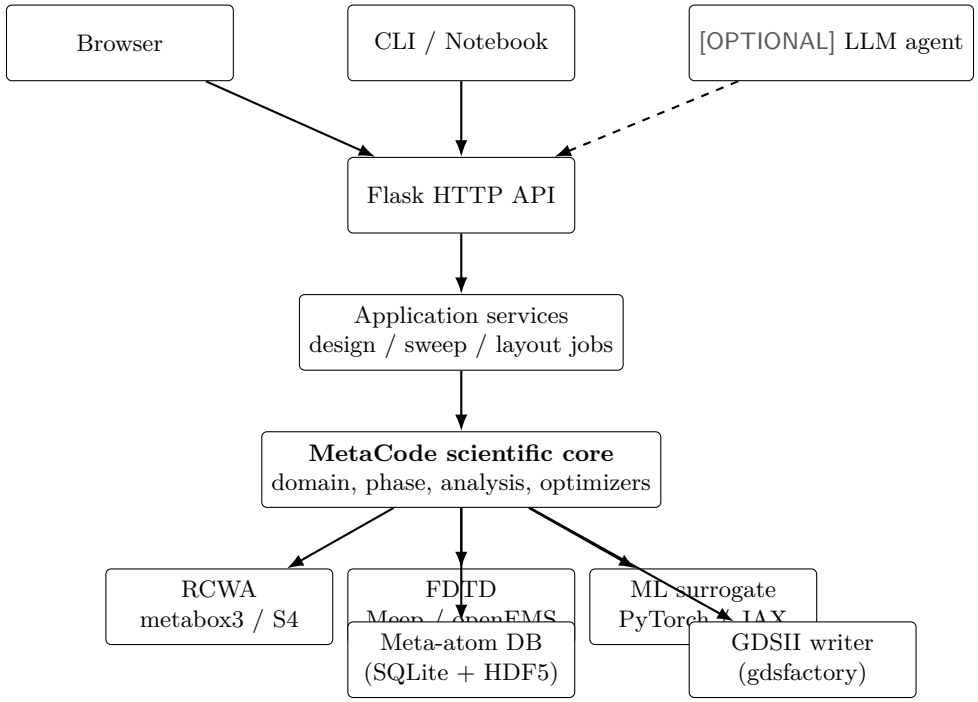


Figure 3.1: Target architecture. Dashed edge = optional later-stage LLM layer. Nothing in the scientific core depends on the LLM.

3.2 Boundaries

The target architecture has five explicit boundaries, each with a contract:

UI ↔ API. JSON over HTTP. Every request is a design or job description that passes Pydantic validation.

API ↔ services. Service classes take validated domain objects, enqueue work, and return job IDs plus artifact handles.

Services ↔ core. Pure function calls into the scientific core. No HTTP, no DB access.

Core ↔ backends. The existing `SimulationBackend` ABC (RCWA) plus a new FDTD interface and a new surrogate interface.

Core ↔ storage. A persistent `ArtifactStore` satisfying the existing `Protocol`, backed by SQLite (metadata) and HDF5 (payloads).

3.3 Design Invariants

1. **No numeric value is generated by a language model.** Every reported number is produced by a solver or a deterministic analysis routine.
2. **Every physics call takes a validated Pydantic object.** Dicts entering the core are rejected until converted.
3. **Every artifact is referenced by a handle.** Large payloads (phase masks, sweep arrays, PSFs) live behind the store; APIs pass handles plus small summaries.
4. **Every long job is asynchronous.** Simulations that take more than a few seconds run as background jobs with progress reporting; the HTTP layer stays responsive.
5. **Every result records provenance.** Backend name, solver version, RCWA harmonics / resolution, timestamp, input hash.

3.4 Where the Current Code Fits

Most of the scientific core is already present; the target architecture is largely a question of filling in the swap-in backends, the persistent store, and the HTTP surface. Table 3.1 enumerates how today's code maps onto tomorrow's architecture.

Table 3.1: Mapping of existing modules to the target architecture.

Target component	**Current source location**
Domain model	`src/metaopticsai/domain/`
RCWA backend	`src/metaopticsai/physics/backends/metabox.py`, `analytical.py`, `s4.py`
Phase library	`src/metaopticsai/phase/`
Analysis	`src/metaopticsai/analysis/`
Optimizers	`src/metaopticsai/optimization/`
GDSII writer	`src/metaopticsai/fabrication/gds.py`
Tool catalogue	`src/metaopticsai/tools/` (16 tools)
Narrow tool catalogue	`mcp_server/tools.py` (6 tools, in-process)
Scaffold LLM agent	`llm/agent.py` (optional future stage)
FDTD backend	*to add* – Meep or openEMS adapter
Persistent store	*to add* – SQLite+HDF5 `ArtifactStore`
HTTP API	*to add* – Flask + Pydantic

Chapter 4

Research Roadmap

4.1 Shape of the Research Programme

The research work supporting MetaCode is sequenced to match the engineering ladder. Each stage has a specific research question whose answer unblocks implementation decisions.

- **Stage 1.** Which equations and references define the problem?
- **Stage 2.** How is a single meta-atom's complex transmission computed to adequate accuracy without commercial software?
- **Stage 3.** How is a validated meta-atom library built and stored?
- **Stage 4.** Which phase profile maps a target optical function (focal length, OAM, Bessel beam) onto the metasurface?
- **Stage 5.** How is the design validated against an independent full-wave solver?
- **Stage 6.** Which inverse-design method is appropriate at a given design budget?
- **Stage 7.** How is the whole pipeline automated, observable, and reproducible?
- **Stage 8/9.** Which agent- or LLM-mediated interfaces make the tool usable to non-expert researchers?
- **Stage 10/11.** What is the first product?

4.2 Stage-to-Paper Mapping

Table 4.1 is the critical cross-reference: for each stage, the papers to read and the algorithms to extract.

4.3 Open-Access Priority

Where more than one reference teaches the same algorithm, the open-access one wins. arXiv or OSA open-access papers are preferable to paywalled journals for an open-source project; students should not have to spend to read the primary sources that back MetaCode.

Table 4.1: Stage → paper → algorithm → MetaCode implementation.

Stage	Papers	Algorithm/concept	MetaCode use
1	[3, 7, 24]	Scalar diffraction; Huygens-Fresnel; generalised Snell's law	Set notation and reference model for every later stage.
2	[17, 13]	RCWA with Lalanne factorisation for TM	Core RCWA engine; informs `xy_harmonics` default.
2	[16]	Fourier modal method with absorbing boundaries	Second, independent RCWA backend.
2	[20, 14]	Yee-grid FDTD	FDTD cross-check and validation backend.
3	[15, 11, 5]	Dielectric Huygens meta-atom libraries	Baseline template sets and reference curves.
4	[7, 11]	Fresnel-zone phase profile; focusing-lens phase	`phase.profiles.fresnel_zone_lens`.
5	[20, 14]	FDTD benchmarking	New backend; FDTD-vs-RCWA agreement test.
6	[18, 22, 8]	Adjoint-based inverse design	Replace heuristic optimiser; extend the gradient path.
6	[9]	NN surrogates for metasurface response	Flesh out `SurrogateOptimizer`.
6	[1]	Bayesian / TPE hyperparameter search	Outer-loop search over materials, periods, heights.
9	[23, 2]	Transformer tool use; MCP tool protocol	Optional LLM-driven agent layer.

4.4 Research Hygiene

Three habits make the research track pay off:

- **Reproduce before extending.** Every new algorithm enters MetaCode only after a notebook reproduces the key result from the paper (or an open-access reference implementation) within a documented tolerance.
- **Cross-check across methods.** No single simulation method is trusted alone. RCWA (metabox / S4) must agree with FDTD (Meep / openEMS) on at least one canonical case per material.
- **Version the data.** Material CSVs, template dicts, and reference FDTD curves live in version control with provenance notes citing the paper they came from.

Chapter 5

Stage 1 – Scientific Foundations

5.1 Objective

Produce a short, internal *Foundations* document and a set of notebooks that re-derive the equations MetaCode depends on from first principles, together with cross-checks against known closed- form limits. This is a cheap, high-impact stage because every later step is easier to debug when the formulas are internally owned.

5.2 Why Necessary

Later stages require unambiguous conventions: units (metres vs. nanometres vs. micrometres), signs (phase accumulation direction), grids (how a radial profile is sampled on a Cartesian pixel grid), polarisation (Jones vector convention). Pinning these early avoids a very common failure mode where two reasonable implementations disagree by a sign or a factor of two.

5.3 Required Theory

- Scalar diffraction and the Huygens-Fresnel principle [3, 7].
- Fresnel-zone focusing: the lens-maker equation in the thin-lens limit and its finite-f generalisation, Eq. (5.1).
- Fraunhofer propagation and its relation to the FFT [7].
- Strehl ratio, Maréchal criterion.
- Noll-indexed Zernike polynomials [19].
- The physical model of a dielectric meta-atom's local complex transmission ([24, 15]).

$$\phi_{\mathrm{FZL}}(r) \;=\; \frac{2\pi}{\lambda}\left(f - \sqrt{r^2 + f^2}\right) \bmod 2\pi. \tag{5.1}$$

5.4 Implementation Tasks

- [NOW] Audit the units at every function boundary in `src/metaopticsai/physics/` and `src/metaopticsai/phase/`. Pin: domain layer in nm; backend layer in metres; analysis in μm.
- [NOW] Write a `notebooks/foundations/` directory with: unit conversion, Huygens-Fresnel derivation, Fresnel vs. Fraunhofer accuracy, Noll Zernike orthogonality check on a dense disk sample.
- [NOW] Document the sign convention used by `analysis/psf.py`: $e^{+i\phi}$ at the pupil, FFT via `np.fft.fftshift(np.fft.fft2(np.fft.ifftshift(.)))`. Record it in `docs/overleaf_latex/` companion report.
- [NEXT] Pin the Jones-vector convention; add a polarisation smoke test to the analytical backend.
- [NEXT] Replace the UTF-16 `requirements.txt` with a UTF-8 `pyproject.toml` and extras: `tf`, `fdtd`, `gds`, `web`, `llm`.

5.5 Expected Inputs and Outputs

- Inputs: existing source, published references.
- Outputs: internal foundations document, notebooks, unit-convention cheat sheet, updated dependency manifest.

5.6 Validation Criteria

- A Noll Z4 (defocus) imprinted on a uniform aperture produces a predictable Strehl drop that matches a closed-form reference within 1%.
- A uniform-phase aperture produces Strehl ratio 1.0 within 10^{-6}.
- A tilted uniform-phase aperture shifts the PSF peak by the expected number of pixels given the pupil tilt.

5.7 Open-Source Tools

NumPy, SciPy, Jupyter, matplotlib. Nothing else is required at this stage.

5.8 Hardware Requirements

A laptop. Every example in this stage runs in seconds on a modest CPU.

5.9 What NOT to Build Yet

No GPU work, no FDTD, no inverse design, no web UI, no LLM.

5.10 Deliverable and Exit Criterion

Deliverable. `notebooks/foundations/` directory, unit-convention cheat sheet, a passing smoke test for each closed-form reference.

Exit criterion. Every example in the foundations notebooks matches its closed-form reference within a written tolerance. Developer confidence that any later mismatch indicates a real bug, not a sign/unit error.

Chapter 6

Stage 2 – Unit-Cell / Meta-Atom Engine

6.1 Objective

Make the single-cell RCWA path reliable, repeatable, and independent of a single vendored solver. The deliverable is a unit-cell response calculator (complex $t_x(d, \lambda)$) that can be queried either by the metabox3-backed RCWA path, by an S4-based RCWA path, or by an FDTD backend (Stage 5), with a documented agreement window between them.

6.2 Why Necessary

Every downstream step – phase library, metalens synthesis, optimisation – is only as good as the unit-cell response. Today, MetaCode has one RCWA engine (metabox3) and an analytical heuristic; moving to a two-engine policy prevents silent errors from a single-source numerical bug.

6.3 Required Theory

- RCWA with the Lalanne factorisation [17, 13] for TM-polarisation convergence.
- Scattering-matrix formulation for layered periodic structures [16].
- Dispersion models: Sellmeier equations for TiO_2, SiN, SiO_2; tabulated $n + ik$ for Si (visible/NIR).

6.4 Algorithms

- Build a parameterised unit cell (pillar on substrate, square period), with the pillar radius as a variable feature.
- For each diameter d_k, compute the zeroth-order complex transmission $t_x(d_k; \lambda)$ via the chosen backend.
- Unwrap the phases: $\varphi_k = \mathrm{unwrap}(\arg t_x(d_k)) - \arg t_x(d_0)$.

- Compute phase coverage cov $= (\varphi_{\max} - \varphi_{\min})/(2\pi)$ and mean transmission $\overline{|t_x|}$.

6.5 Implementation Tasks

- [NOW] Lift the fixed-n dict in `MetaboxBackend` into a call to `physics.materials.get_material_index(wl_nm)` so sweeps respect dispersion.
- [NEXT] Add an S4Backend implementation using [16]; keep its `is_available()` honest.
- [NEXT] Add a sweep-axis dispatch so that `sweep_param` $\in$ {`"diameter"`, `"height"`, `"period"`, `"wavelength"`} at least for the metabox backend.
- [LATER] Build a `compare_backends(params)` helper that runs the same sweep through two backends and reports pointwise phase/amplitude agreement.
- [LATER] Replace the heuristic `amplitude` model in `AnalyticalBackend` with a thin-film reference when a trusted analytical limit exists (e.g. sparse pillar array, isolated-scatterer approximation).

6.6 Inputs / Outputs

- Inputs: validated `MetalensDesignParams`, `IncidenceSpec`, sweep axis + values.
- Outputs: `RCWASweepResult`, backend name, elapsed time, provenance dict.

6.7 Validation Criteria

- Phase monotonicity: no non-trivial fraction of the sweep produces $\mathrm{d}\varphi/\mathrm{d}d < 0$ in a regime where it is expected monotone.
- Cross-backend agreement: pointwise RCWA phase agreement between metabox3 and S4 within 5% of 2π at matching sampling.
- Sanity: a disc of diameter approaching the period must either saturate or show a documented failure mode (not a silent spurious 2π wrap).

6.8 Papers

- Moharam et al. [17] – the stable RCWA formulation.
- Lalanne and Morris [13] – correct TM convergence.
- Liu and Fan [16] – S4 is the independent reference solver.

6.9 Open-Source Tools

- S4 (https://github.com/victorliu/S4) – validation RCWA, independent of metabox3.
- metabox3 (vendored) – primary RCWA.
- NumPy/SciPy – interpolation, unwrapping.

6.10 Hardware Requirements

Laptop CPU is adequate for small sweeps. GPU for metabox3 helps larger sweeps but is optional at this stage.

6.11 What NOT to Build Yet

No full-aperture lens simulation, no inverse design, no persistent database.

6.12 Deliverable and Exit Criterion

Deliverable. A two-backend RCWA sweeper plus a `compare_backends` notebook with cross-plots.

Exit criterion. Phase/amplitude cross-plots between metabox and S4 fall within the documented tolerance for the three current templates.

Chapter 7

Stage 3 – Meta-Atom Database and Phase Library

7.1 Objective

Turn one-off sweeps into a queryable, versioned *meta-atom database* and a reusable *phase-to-geometry* inverse table. Given a target local phase φ^* at wavelength λ, the database must deliver the pillar diameter d^* that realises it, plus the expected transmission.

7.2 Why Necessary

A fast, deterministic $\varphi \to d$ map is the backbone of every metalens layout. Without it, every design request re-runs an RCWA sweep for identical geometry. With it, the dominant cost of a design moves from physics to disk I/O.

7.3 Required Theory

- Monotone cubic interpolation (SciPy's `interp1d` with `kind="cubic"`; MetaCode already uses this pattern for the FDTD library).
- Quantisation of a continuous phase mask into discrete levels (`phase.quantization.quantize`).
- Chen *et al.*'s dielectric Huygens meta-atom analysis [5] and the TiO_2 pillar analysis of Khorasaninejad *et al.* [11].

7.4 Database Schema (SQLite)

A small, versioned schema. The payload (phase/amplitude arrays) lives in HDF5 outside the DB; the DB is a searchable index.

```
CREATE TABLE sweep (
  id INTEGER PRIMARY KEY,
  material TEXT NOT NULL,
  shape TEXT NOT NULL, -- cylinder | cross | fin
```

```
  wavelength_nm REAL NOT NULL,
  height_nm REAL NOT NULL,
  period_nm REAL NOT NULL,
  d_min_nm REAL NOT NULL,
  d_max_nm REAL NOT NULL,
  n_samples INT NOT NULL,
  backend TEXT NOT NULL, -- metabox|s4|analytical|fdtd
  solver_rev TEXT NOT NULL,
  input_hash TEXT NOT NULL, -- repro key
  h5_path TEXT NOT NULL, -- diameters, amplitudes, phases
  created_at TEXT NOT NULL
);
CREATE INDEX ix_sweep_lookup
  ON sweep(material, wavelength_nm, period_nm, height_nm);
```

Listing 7.1: Proposed SQLite schema.

7.5 Implementation Tasks

- [NOW] Add a `src/metaopticsai/db/` package with an `ArtifactStore` that satisfies the existing `Protocol` and persists to SQLite + HDF5.
- [NEXT] Wrap `sweep_diameter` with a cache that checks the DB by `input_hash` before running physics.
- [NEXT] Add a `phase_to_diameter(sweep_id, phi_rad)` helper that fits a monotone cubic to the stored sweep.
- [LATER] Pre-populate the DB with the three templates plus FDTD reference curves (five wavelengths already in `physics/fdtd.py`).
- [LATER] Add a CLI `metacode db ingest` that reproduces a sweep from a template and stores it.

7.6 Inputs / Outputs

- Inputs: a validated template; a chosen backend; a sweep-axis spec.
- Outputs: an `sweep_id` that any caller can resolve back to arrays via the store.

7.7 Validation Criteria

- A re-request with identical inputs returns an existing `sweep_id` without re-running physics.
- `phase_to_diameter` inverted and re-forwarded agrees with the stored sweep within $\mathcal{O}(10^{-3})$ radians at the sample points.
- A deliberate change to `solver_rev` invalidates the cache (new row inserted).

7.8 Papers

- Chen *et al.* [5] – which Huygens meta-atom families are worth tabulating first.
- Khorasaninejad *et al.* [11] – the TiO_2@532 nm reference curve.

7.9 Open-Source Tools

- SQLite (bundled with Python's `sqlite3`).
- HDF5 via `h5py`.
- SciPy `interp1d`.

7.10 Hardware Requirements

Laptop. The DB is a few MB for 100 sweeps.

7.11 What NOT to Build Yet

Multi-tenant DB, cloud storage, access control. SQLite with a single-user file is sufficient throughout Stages 3–7.

7.12 Deliverable and Exit Criterion

Deliverable. A persistent store that returns `sweep_id`s and lets any later stage look up phase/diameter tables.

Exit criterion. A metalens-layout job (Stage 4) can be driven end-to-end from a cached sweep without invoking any RCWA engine.

Chapter 8

Stage 4 – Metalens Layout and Full-Aperture Simulation

8.1 Objective

Produce a full-aperture metalens layout from a target focal length, aperture, and material; validate the layout with a near-/far-field simulation; export the result as a GDSII file.

8.2 Why Necessary

The three previous stages produce a unit-cell library. Stage 4 is where "design" happens: tiling the library onto a focusing phase profile, quantising, and estimating the lens's optical performance.

8.3 Required Theory

- Phase profile for a focusing lens (Eq. 5.1).
- Angular-spectrum propagation (available via `MetaboxBackend.propagate_field`; equivalent to Fresnel/Fraunhofer in the appropriate limit [7]).
- PSF and Strehl ratio for the aberrated aperture ([3, 7]; present in `analysis/strehl.py`).

8.4 Design Pipeline

1. Choose template ⇒ validate parameters.
2. Fetch (or compute) the matching sweep from the DB.
3. Build a focusing phase profile at the sampling matched to the unit-cell period (`pixel_size_um = period_nm / 1000`).
4. Quantise to N levels (default 8).
5. Map each level to its diameter via the DB's phase-d interpolant.

6. Lay out pillars with the GDSII writer.

7. Propagate the phase mask (modulo the amplitude attenuation implied by the sweep's $|t_x|$) to the focal plane; report PSF, Strehl, FWHM, encircled-energy radii, top-5 Zernike coefficients.

8.5 Implementation Tasks

- [NOW] Add a `design_metalens(params, sweep_id, n_levels)` service that drives the pipeline above and returns `{design_handle, psf_handle, gds_handle, metrics}`.
- [NEXT] Make the propagation call optional; the analytical Fraunhofer fallback already in `analysis/psf.py` is adequate for pre-visualisation.
- [NEXT] Add a per-pixel amplitude map by looking up $|t_x|$ for each quantised diameter and multiplying it into the pupil before PSF/MTF.
- [LATER] Add a tolerance scan: perturb pillar diameters by ± 10 nm (typical process variance) and re-evaluate the PSF.

8.6 Inputs / Outputs

- Inputs: validated `MetalensDesignParams`, `sweep_id`, output name.
- Outputs: GDSII file in `outputs/`, `design_handle` + metrics report.

8.7 Validation Criteria

- For the `tio2_metalens_532nm` template, Strehl after amplitude weighting is ≥ 0.5 (the simple FZL target).
- FWHM is within 20% of the diffraction limit $\lambda/(2\mathrm{NA})$.
- The exported GDSII opens cleanly in KLayout [12] and the pillar count matches the expected $\pi R^2/p^2$ within the circular-aperture rounding.

8.8 Papers

- Goodman [7] – propagation and PSF.
- Khorasaninejad *et al.* [11] – reference numbers for Strehl, FWHM at 532 nm.
- Chen *et al.* [6] – dispersion-engineered flat optics; sets expectations for later chromatic work.

8.9 Open-Source Tools

- `gdsfactory` [10] – the GDSII writer MetaCode already uses.
- KLayout [12] – visual verification of exported masks.

8.10 Hardware Requirements

Laptop CPU is adequate for apertures up to a few hundred microns; the full-aperture propagation stays within an FFT whose grid is `N_pad = N  4`, with N set by `diameter_um / pixel_size_um`.

8.11 What NOT to Build Yet

Full 3-D vector field in a volumetric mesh; multi-wavelength chromatic composition; wavefront sensing / closed-loop measurement.

8.12 Deliverable and Exit Criterion

Deliverable. A one-call service `design_metalens(...)` that produces a GDSII and a metrics report from a template.

Exit criterion. Each of the three built-in templates produces a report whose Strehl/FWHM lies within the written tolerance; the GDSII loads in KLayout.

Chapter 9

Stage 5 – Electromagnetic Validation Backend

9.1 Objective

Add a second, independent full-wave simulation path (FDTD) that can validate selected unit cells and small lens patches against the RCWA engine. This is the *trust* stage: no later number is publishable without a path from design through at least one alternative solver.

9.2 Why Necessary

RCWA is efficient but assumes an infinite periodic structure. A finite metalens has edge effects, substrate interactions, and oblique excitation; the only way to catch RCWA errors is to run the same geometry on a different solver. Meep [20] and openEMS [14] are the open-source candidates.

9.3 Required Theory

- Yee-grid FDTD and the Courant stability condition.
- Perfectly matched layer (PML) absorbing boundaries.
- Convergence as a function of grid spacing and PML depth.
- Dispersive materials via Drude / Lorentz / tabulated $n(\lambda)$.

9.4 Backend Interface

Reuse the existing `SimulationBackend` ABC. Add a Meep backend behind the same methods: `simulate_unit_cell`, `sweep_unit_cells`, `propagate_field`. Meep's Python API is pure Python and installable via `conda-forge` or `pip` depending on platform; the adapter should degrade gracefully when Meep is missing (identical pattern to the current `MetaboxBackend`).

9.5 Implementation Tasks

- [NOW] Add `physics/backends/meep.py` with the three ABC methods. Start with `simulate_unit_cell` only; return `NotImplementedError` on the sweep and propagation methods initially.
- [NEXT] Add a notebook that reproduces one TiO_2 pillar's $t_x(d)$ curve on Meep and overlays metabox3's curve. Record the convergence spacing.
- [LATER] Extend `sweep_unit_cells` to batched Meep runs; the batching will be slower than metabox3 but is only used for validation.
- [LATER] Add openEMS as an alternative open-source FDTD; the two FDTD codes should agree with each other and with S4 to within documented tolerances.

9.6 Inputs / Outputs

Same as the RCWA backends; the Meep backend returns the same `UnitCellResult`/`RCWASweepResult`/`FieldResult` dataclasses.

9.7 Validation Criteria

- For a canonical pillar, metabox3 and Meep agree in phase within 10% of 2π at matched sampling.
- For a canonical far-field, Meep and metabox3's angular-spectrum propagation agree on peak location and FWHM within 10%.
- Grid-refinement study shows the Meep results have converged (change $<$ chosen tolerance when the grid is halved).

9.8 Papers

- Oskooi *et al.* [20] – Meep formulation and API.
- Liebig *et al.* [14] – openEMS as a second open FDTD.

9.9 Open-Source Tools

- Meep (`https://meep.readthedocs.io/`).
- openEMS (`https://www.openems.de/`).

9.10 Hardware Requirements

Laptop CPU is sufficient for isolated unit cells; a cheap cloud instance (16 GB RAM, 8 vCPU) for small-area patches. No GPU needed.

9.11 What NOT to Build Yet

Full-aperture lens FDTD (10^{10}-cell grids for a 1 mm aperture are not practical anywhere this document discusses).

9.12 Deliverable and Exit Criterion

Deliverable. A working Meep backend that answers `simulate_unit_cell` within the agreed tolerance of metabox3 for one template per material.

Exit criterion. A `cross_check(template)` CLI emits a pass/fail report for every shipped template.

Chapter 10

Stage 6 – Optimization and Inverse Design

10.1 Objective

Replace the current placeholder heuristic optimiser with a working inverse-design path based on adjoint-method gradients (via the existing metabox3 TensorFlow path) and a Bayesian outer loop (via Optuna [1]). Flesh out the surrogate optimiser once a neural approximation is trained on the DB.

10.2 Why Necessary

The current `HeuristicOptimizer` is a scoring function. A real design workflow needs genuine search over $(d_{\min}, d_{\max}, h, p)$ for a target Strehl, FWHM, or an application-specific FOM.

10.3 Required Theory

- Adjoint-method inverse design in nanophotonics [18, 22, 8].
- Bayesian / TPE outer-loop optimisation [1].
- Neural surrogates for metasurface response [9].

10.4 Three Layers of Optimisation

Local (adjoint). Already present as `GradientOptimizer` wrapping metabox3's `optimize_single_lens_assembly`. Harden it: catch metabox3 API drift (already partially done), surface the loss history, seed TF randomness.

Outer (Bayesian). Implement a new `OptunaOptimizer` that samples $(h, p, d_{\min}, d_{\max})$, triggers a sweep + a lens-design, and reports a scalar FOM. Trial budget configurable; parallelism via Optuna's `n_jobs`.

Learned (surrogate). Train a small MLP (or GP) that maps $(d, h, p, \lambda, \text{material}) \to (|t|, \varphi)$ from the DB. Replace the inner RCWA loop in `OptunaOptimizer` with the surrogate once its RMSE on a hold-out set is below a documented threshold.

10.5 Implementation Tasks

- [NOW] Harden `GradientOptimizer`: expose `tf.random.set_seed`; record loss history as a dataclass; dump a provenance JSON next to the design.
- [NEXT] Add `optimization/optuna_outer.py` with an `OptunaOptimizer` that satisfies the existing `Optimizer` ABC.
- [LATER] Train a surrogate on the DB; implement `SurrogateOptimizer` by loading the surrogate and using it inside the Optuna inner evaluation.
- [LATER] Add an application-aware FOM registry (point-source Strehl, extended-source MTF50, achromatic composite) so the optimiser can target different problem classes without rewiring.

10.6 Validation Criteria

- Adjoint path: for a toy design where the global optimum is known analytically, the optimiser's FOM gets within 5% in $\leq$50 iterations.
- Outer loop: Optuna's best trial beats a 20-point random search baseline on the same budget.
- Surrogate: RMSE on a held-out sweep $< 5\%$ of 2π in phase; < 0.05 in amplitude.

10.7 Papers

- Molesky *et al.* [18] – overview of inverse design.
- Pestourie *et al.* [22] – large- area metasurface inverse design.
- Hughes *et al.* [8] – Maxwell-equation differentiation in modern frameworks.
- Jiang *et al.* [9] – neural surrogates.
- Akiba *et al.* [1] – Optuna.

10.8 Open-Source Tools

JAX [4], PyTorch [21], Optuna [1], scikit-learn.

10.9 Hardware Requirements

CPU for Optuna outer loop; occasional GPU for the adjoint inner loop (TF-backed). Surrogate training is cheap on CPU for small MLPs.

10.10 What NOT to Build Yet

Topology-optimised freeform meta-atoms (large design spaces); end-to-end differentiable image-formation co-design; multi- objective Pareto exploration. These are reasonable Stage 12+ extensions.

10.11 Deliverable and Exit Criterion

Deliverable. A working outer-loop optimiser (Optuna) plus hardened adjoint and seedable TF randomness.

Exit criterion. On a canonical design request, the optimiser improves Strehl over the heuristic baseline by a measurable margin on a 200 µm aperture.

Chapter 11

Stage 7 – Automated Design Pipeline

11.1 Objective

Compose the previous stages into a one-call automated workflow: given a design request (natural-language or JSON), produce a report containing a GDSII, a metrics summary, provenance, and references back to the backend outputs.

11.2 Why Necessary

Research notebooks are not products. A dependable pipeline that runs non-interactively, reports every number it produced, and leaves a reproducible trail is the entrance ticket to any Stage 10 web product.

11.3 Pipeline

11.4 Implementation Tasks

- [NOW] Introduce a `jobs/` module with a plain thread-backed job manager (the broader scientific tool layer already expects an injected manager). Keep it swappable with a Celery / RQ backend later.
- [NEXT] Add `design_metalens_job(request) -> job_id` that runs the pipeline asynchronously and streams status.
- [NEXT] Add a `Report` dataclass with structured fields: input hash, template name, sweep ID, achieved metrics, GDSII path, timestamps, backend versions.
- [LATER] Add retries with exponential backoff for flakey solver failures; abort with a documented error after n retries.
- [LATER] Add a `metacode run <request.json>` CLI that wraps the job manager for scripting.

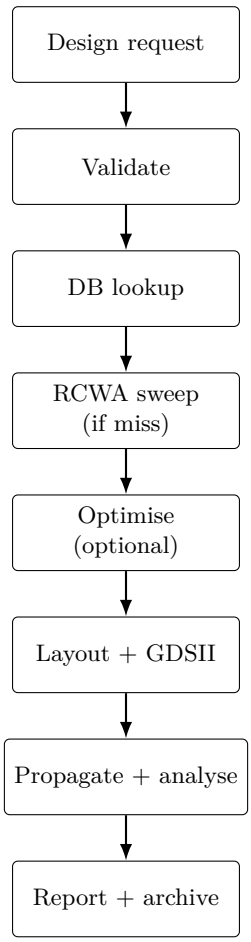


Figure 11.1: The automated design pipeline. Every node emits an artifact handle; the report chains them into a reproducible trail.

11.5 Observability

- Structured logs (one JSON object per event) using the existing `utils.logging` setup.
- A per-job directory under `outputs/jobs/<id>/` containing: `request.json`, `report.json`, `design.gds`, `psf.png`, `mtf.png`.
- Timing instrumentation around every backend call.

11.6 Validation Criteria

- Replaying the saved `request.json` on a fresh checkout reproduces the same metrics within a documented tolerance.
- A deliberately malformed request returns a clean error object with a `field` path, never a stack trace.

11.7 Deliverable and Exit Criterion

Deliverable. An asynchronous, observable, reproducible design pipeline reachable via Python API and CLI.

Exit criterion. A fresh checkout can `metacode run request.json` and reproduce a prior job's metrics.

Chapter 12

Stage 8 – Agent and Tool Orchestration

12.1 Objective

Expose the tool catalogue in `src/metaopticsai/tools/` over a transport-neutral surface: a Python registry (today), a Flask HTTP API (Stage 10), and – optionally – an MCP stdio server for future LLM clients.

12.2 Why Necessary

The scientific-tool catalogue is already present with 16 tools and Pydantic schemas, but no shipping transport binds it to clients. Automation (Stage 7) uses the Python registry directly; the web platform (Stage 10) will need an HTTP surface.

12.3 Implementation Tasks

- [NOW] Add `src/metaopticsai/tools/store/` with a concrete `SQLiteHDF5Store` implementing the `ArtifactStore Protocol`.
- [NEXT] Add `src/metaopticsai/http/` with a Flask blueprint that mirrors each tool's Pydantic input/output onto a route. Keep the blueprint registration driven by `build_tool_registry`.
- [LATER] Add an optional stdio MCP server (`fastmcp` is already in `requirements.txt`) that wraps the same registry. Treat this as independent from the Flask layer; both share the registry.

12.4 Deliverable and Exit Criterion

Deliverable. A Python API, a Flask HTTP API, and an optional stdio MCP server all driven by one tool registry.

Exit criterion. A single integration test calls each tool via each transport and asserts matching output dicts.

Chapter 13

Stage 9 – Optional LLM Interface

13.1 Objective

Add an optional, replaceable conversational interface that turns natural-language design requests into validated tool calls over the Stage 8 registry. This stage is strictly optional; MetaCode must function fully without it.

13.2 Why This Stage Is Optional

Scientific users are generally comfortable with structured inputs; natural language is a convenience, not a necessity. Research budgets rarely justify a dependence on a paid API, and model providers change terms unpredictably. The decision is therefore: build the entire scientific engine so that an LLM is a sweetener, never a load-bearing component.

13.3 Design

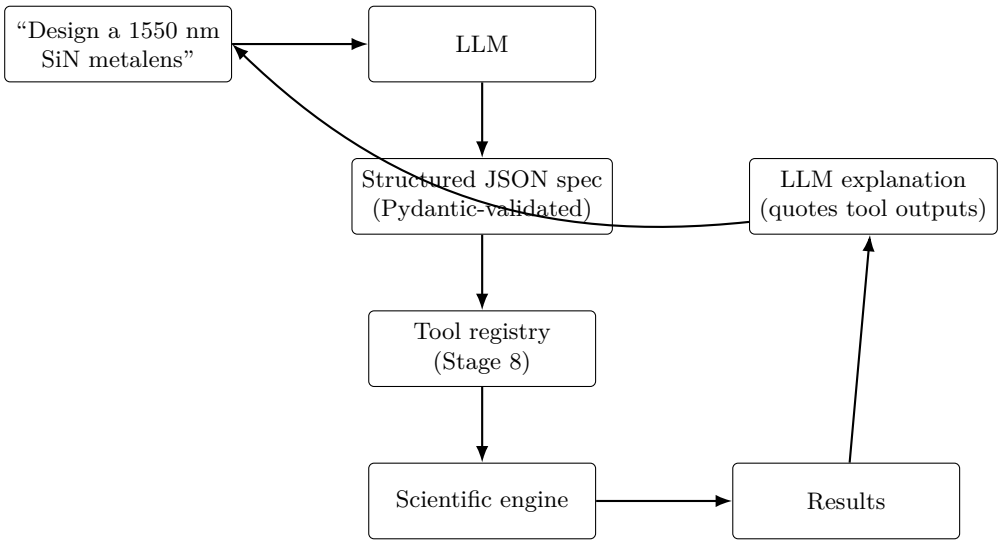


Figure 13.1: Optional LLM interface. The language model only produces or edits a structured JSON specification; numeric claims in the final explanation must come from the tool outputs.

13.4 Providers

Any Groq-hosted tool-calling model is a candidate (the current repository already contains an exploratory client wrapper). A local alternative – a quantised open-weights model served via `llama.cpp` or vLLM – is a strong fallback for air-gapped or cost-constrained deployments. The abstraction must make the provider swappable; no code outside the LLM package should assume Groq.

13.5 Safety and Determinism

- **Never trust numeric output from the model.** Every number in the final explanation must appear verbatim in a tool call's result.
- **Pydantic the model's JSON.** The structured design spec goes through `MetalensDesignParams` before it can trigger a sweep.
- **Bounded loops.** Cap at a small number of tool round-trips (the current exploratory agent clamps to 5).
- **Record usage.** Capture `usage.total_tokens` per request and attribute cost per user.
- **Fail open to the structured form.** If the model is unavailable, the UI must still accept a JSON spec and run the same pipeline.

13.6 Implementation Tasks

- [LATER] Factor the Groq client into a provider-neutral `ChatProvider` interface. Add adapters for Groq and for a local `llama.cpp` HTTP server.
- [LATER] Replace the hard-coded system prompt with a templated prompt plus a reproducible seed and recorded prompt hash.
- [LATER] Add a numeric-faithfulness check: post-generation, re-parse the model's summary and verify every number appears in some tool-call result; downgrade to a warning banner otherwise.
- [LATER] Hook the agent into the tool registry over HTTP (Stage 8), not through an in-process `dispatch`; the UX should be identical for notebook, CLI, and web client.

13.7 Papers

- Vaswani *et al.* [23] – Transformer foundations.
- Anthropic MCP specification [2] – tool/transport shape for any serious agent product.

13.8 What NOT to Build Here

- **No autonomous optimisation.** The LLM must not launch open-ended Optuna budgets; those are triggered by structured requests.
- **No RAG over the whole literature.** A small, curated prompt library is enough for Stage 9; a vector database is Stage 12+.
- **No LLM-as-reviewer.** Validation runs on the scientific engine, not on the model's self-assessment.

13.9 Deliverable and Exit Criterion

Deliverable. A provider-neutral chat adapter plus a numeric-faithfulness check, both optional and off by default.

Exit criterion. The web platform (Stage 10) can offer a "describe your lens" box that routes through the chat adapter when enabled and transparently falls back to the structured form otherwise.

Chapter 14

Stage 10 – Validation Programme

14.1 Objective

Build a reusable, repeatable validation suite that cross-checks simulation and design outputs against open-access literature, alternative solvers, and – for a handful of canonical cases – measurements where accessible.

14.2 Why Necessary

A photonics design tool without a validation story is unusable for publication-grade work. The validation suite is a prerequisite for anyone outside the team to trust MetaCode's outputs.

14.3 Validation Classes

Closed-form ("level 0"). Uniform aperture Strehl = 1; defocus Z4 Strehl drop vs. Maréchal; Fraunhofer PSF FWHM = $\lambda/(2\mathrm{NA})$ to leading order.

Cross-solver ("level 1"). metabox3 vs. S4 vs. Meep on three canonical unit cells per shipped material.

Literature ("level 2"). Reproduce one figure per cited paper (e.g. [11] TiO_2 phase curve at 532 nm) and store the overlay plot in `docs/`.

Measurement ("level 3", aspirational). Where a collaborator has fabricated and characterised a device, compare measured vs. simulated focal spot.

14.4 Implementation Tasks

- [NOW] Add `tests/golden/` with level-0 cases; wire them into a plain `pytest` run.
- [NEXT] Add `tests/cross_solver/` with level-1 cases; skip gracefully when a solver is absent.
- [LATER] Add `tests/literature/` with level-2 overlays; the plots become part of the automatically built docs site.

- [LATER] Add CI (GitHub Actions) that runs level-0 and level-1 on push; level-2 nightly.

14.5 Deliverable and Exit Criterion

Deliverable. A validation suite that any contributor can run locally with `pytest` and that CI runs on every push.

Exit criterion. Level-0 is 100% green on main. At least two level-1 cases pass per shipped material.

Chapter 15

Stage 11 – Productisation

15.1 Objective

Promote the automated pipeline (Stage 7), the tool catalogue (Stage 8), and the validation suite (Stage 10) into a narrow, usable product. This is the first stage where "users" are a real population rather than contributors.

15.2 Narrow Scope

The product does *one thing*: generate a fabrication-ready metalens layout and a performance report, from a parameter form, for one of a short list of templates. It does not try to be a general photonics CAD system.

15.3 Who the First Users Are

- Graduate students in photonics/nanoscience groups.
- Postdocs preparing fabrication runs who need a quick, disposable layout for an exploratory experiment.
- Teaching assistants running a nanophotonics course and needing reproducible class examples.

Early users are the authors of the next paper, not the paying customers of a cloud SaaS; the product earns its right to grow by being useful to them.

15.4 Product Surface

- A single web form (Stage 10 UI) accepting the fields of `MetalensDesignParams` plus a template selector.
- A "run" button that queues a job.
- A result page with: metrics, PSF/MTF plots, a GDSII download link, and a shareable job URL.

- A history page listing the user's previous jobs.

15.5 Non-Features

- Multi-user teams, SSO, roles. Add when a paying user asks for them.
- Payment / subscription. Free for the first wave.
- Fabrication-house integration (Direct DRC submission). Add once a design has been fabricated at least once.
- Chat-based interface. Optional, Stage 9.

15.6 Deliverable and Exit Criterion

Deliverable. A deployable Flask app that invokes the Stage 7 pipeline from a form, backed by Stage 8's persistent store.

Exit criterion. Three outside users complete a design end-to-end and the GDSII loads in KLayout.

Chapter 16

MVP Architecture

16.1 Target MVP

[MVP] The minimal viable product is a Flask web application backed by the MetaCode scientific engine, SQLite, and HDF5. It runs on a single \$5–\$10/month CPU VM and serves researchers who want a quick metalens layout.

16.2 One-Picture Architecture

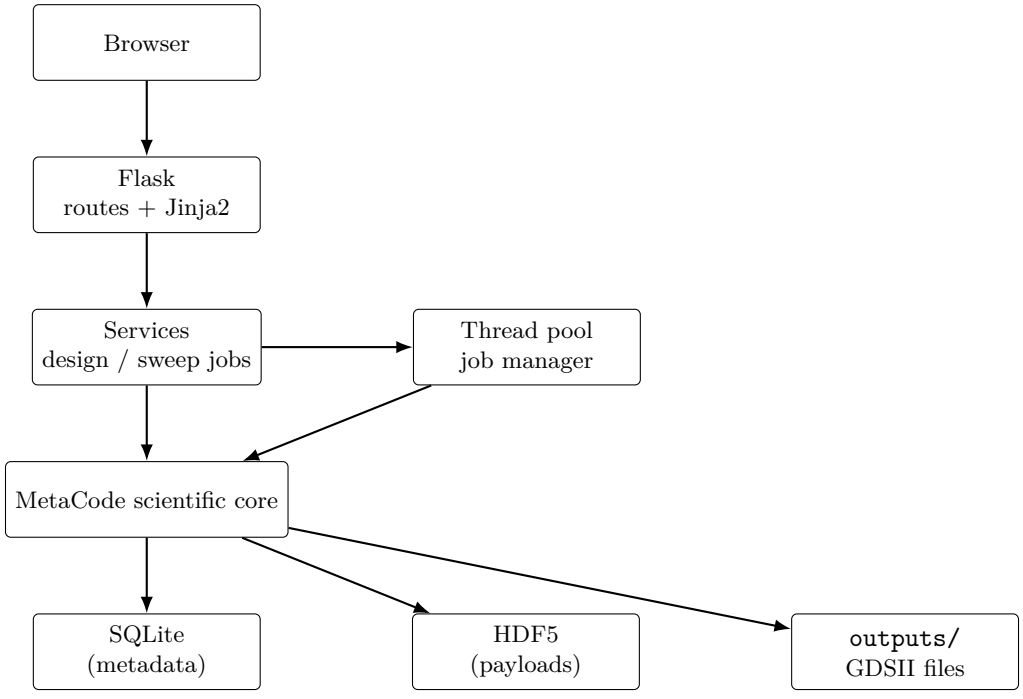


Figure 16.1: MVP architecture. Everything runs on one VM; the job manager is a thread pool, not Celery; storage is SQLite + HDF5 + `outputs/`.

16.3 Folder Structure

```
mvp/
|-- app.py <- Flask app factory + routes
|-- services/
| |-- design_service.py <- orchestrates sweep + layout
| `-- jobs.py <- thread pool job manager
|-- templates/
| |-- base.html
| |-- index.html <- design form
| |-- job.html <- job status + results
| `-- history.html <- user's past jobs
|-- static/
| |-- css/main.css
| `-- js/app.js <- small vanilla JS, no framework
|-- db/
| |-- schema.sql
| `-- store.py <- SQLite + HDF5 ArtifactStore
`-- tests/
    `-- test_mvp.py
```

Listing 16.1: MVP folder structure.

The scientific logic lives in the existing `src/metaopticsai/` package; `mvp/` is purely glue.

16.4 Flask Routes (Minimum)

Table 16.1: MVP Flask routes.

Method	Route	Purpose
GET	`/`	Design form, template selector.
POST	`/design`	Submit a design, create job.
GET	`/job/<id>`	Job status page; polls while running.
GET	`/job/<id>.json`	JSON status + handles.
GET	`/download/<id>/<kind>`	Download GDSII / PNG / report.
GET	`/history`	List previous jobs (cookie-scoped).
GET	`/healthz`	Liveness probe.

16.5 Database Schema (SQLite)

Two tables are enough for MVP:

```
CREATE TABLE job (
  id TEXT PRIMARY KEY,
  user_cookie TEXT,
  state TEXT NOT NULL, -- queued|running|done|failed
  request TEXT NOT NULL, -- JSON of validated params
  started_at TEXT,
  finished_at TEXT,
  error TEXT
);
```

```
CREATE TABLE artifact (
  handle TEXT PRIMARY KEY,
  job_id TEXT NOT NULL,
  kind TEXT NOT NULL,
  path TEXT NOT NULL, -- h5:// or outputs/jobs/<id>/foo.gds
  metadata TEXT, -- JSON
  created_at TEXT NOT NULL,
  FOREIGN KEY(job_id) REFERENCES job(id)
);
```

16.6 Deployment Target

The whole MVP fits on a $5–$10/month VM (DigitalOcean, Hetzner, OVH, Fly.io, Scaleway). No Kubernetes, no managed Postgres, no object storage. TLS via Caddy or Nginx + Let's Encrypt. Backups: a nightly `tar` of the `data/` directory, pushed to any S3-compatible bucket.

16.7 Observability

- Structured logs to stdout; the host's log aggregator (or a local `journalctl`) captures them.
- A single `healthz` route that pings the DB and returns the solver's `is_available()` report.
- Per-job `outputs/jobs/<id>/report.json` with every number the pipeline produced.

16.8 What the MVP Does Not Do

Enumerated so the roadmap can defend scope against pressure:

- No account system; users are tracked by a cookie.
- No chat interface; a separate form.
- No inverse-design search by default (opt-in with a budget cap).
- No live FDTD; FDTD validation runs offline.
- No team sharing beyond pastable job URLs.
- No billing.

Chapter 17

MVP Implementation Plan

17.1 Order of Work

The MVP build order follows a sharp principle: *do not touch the scientific core*. Every MVP task is composition, not redesign.

1. [MVP] Stand up a Flask app factory in `mvp/` with one route: `GET /` serving a simple form.
2. [MVP] Wire `POST /design` to Pydantic-validate the submitted dict and call the Stage 7 `design_metalens_job(request)`.
3. [MVP] Implement the thread-pool job manager in `mvp/services/jobs.py`.
4. [MVP] Add the SQLite+HDF5 store in `mvp/db/store.py` and plug it in as the `ArtifactStore` injected into `build_tool_registry`.
5. [MVP] Add `GET /job/<id>` and `GET /job/<id>.json` with a trivial polling pattern (meta-refresh or fetch every 2 s).
6. [MVP] Add download routes and a `history` cookie-scoped list.
7. [MVP] Add minimal tests: form submission end-to-end; healthz; download of a known job.
8. [MVP] Deploy to a $5/month VM with Caddy + TLS.
9. [MVP] Invite three outside users; iterate on the form wording and error messages.

17.2 Minimum Feature Set

17.3 Explicitly Not in the MVP

- Accounts, SSO, teams, roles.
- Payment.
- LLM/chat interface.
- Inverse design (Stage 6) exposed by default.

Table 17.1: MVP feature set.

Feature	**Scope**
Template selector	Dropdown over the shipped templates.
Parameter form	Fields of `MetalensDesignParams` with safe defaults.
Design submission	POST a validated request, enqueue a job.
Job status page	Polls `job.json`; shows metrics as they land.
PSF/MTF images	Rendered via `utils.images`; served as PNG.
GDSII download	Served from `outputs/jobs/<id>/`.
Report JSON	The full provenance of the job.
History page	Cookie-scoped list of a visitor's jobs.
Health probe	`/healthz` returning solver availability.

- FDTD (Stage 5) exposed by default.
- Non-metalens elements (axicon, SPP) in the form.
- Multiple output formats beyond GDSII.

Every one of these is a reasonable post-MVP addition. Shipping without them is the point.

17.4 Build Order Diagram

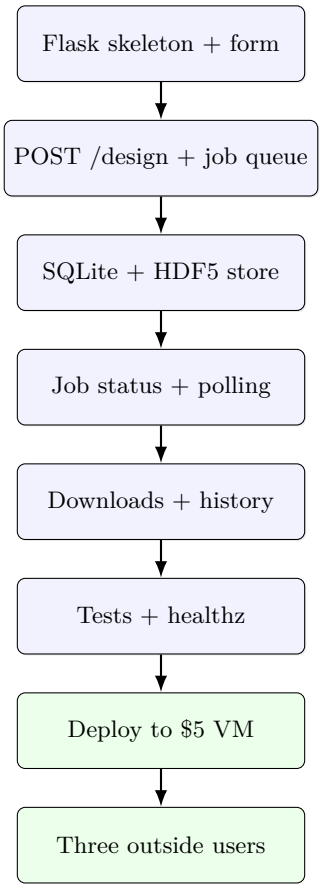


Figure 17.1: MVP build order.

Chapter 18

Low-Cost Infrastructure

18.1 Compute Budgets

The roadmap assumes an aggressively small compute budget. Every stage is designed to work on hardware a graduate student can access without a grant.

Table 18.1: Compute budgets per development phase.

Phase	**Where**	**Typical cost**
Local development (Stages 1–4)	Developer laptop (CPU, 16 GB)	$0
FDTD smoke tests (Stage 5)	Same laptop; small cells	$0
Occasional GPU for metabox3 (Stage 6)	Colab free tier / Kaggle / local 1-GPU dev box	$0–$10/month
MVP production (Stage 11)	$5–$10/month single VM	$5–$10/month
Nightly CI (Stage 10)	GitHub Actions free tier	$0
Backups	Hetzner Storage Box / Backblaze B2	$1–$3/month

18.2 Precompute, Cache, Approximate

Three disciplines keep the production bill at near-zero cost.

Precompute. Ship a precomputed sweep for each template and each supported wavelength. Users of the MVP hit the cache the first time; no RCWA runs on the production host for the shipped templates.

Cache. The DB (Stage 3) hashes `MetalensDesignParams` and the backend version; every repeated request is a disk read.

Approximate. The analytical backend's role is not scientific; it is to keep the MVP responsive when the user's parameters miss the cache and the host is CPU- only. The resulting metrics are flagged as such.

18.3 Where GPUs Help – And Where They Don't

- **Help:** Stage 6 adjoint gradient loops. Each evaluation includes a batched RCWA solve; a single midrange GPU shrinks a day of CPU work to an hour.
- **Help marginally:** Stage 3 metamodel training; a small MLP (10^3–10^4 params) trains in minutes on CPU.
- **Don't help:** The MVP itself. Every production request is a DB read, a Fraunhofer FFT, and a GDSII write. No GPU in sight.

18.4 Local Development on a Laptop

A single `conda env create -f environment.yml` should produce a working stack with:

- Python 3.11+.
- NumPy, SciPy, matplotlib, Pillow, h5py.
- TensorFlow (optional; required for metabox3).
- `S4`, Meep (optional; required for cross-validation).
- `gdsfactory` (optional; required for GDSII).
- Flask, Jinja2, Pydantic, Pydantic-settings.

18.5 Cloud Experimentation: Zero-Cost Options

- **Google Colab (free tier).** 12-hour runs on a shared T4; adequate for metabox3 sweeps and small adjoint runs. Save results to Drive.
- **Kaggle Kernels.** 30 hours/week of GPU time; strong for Stage 6 experiments.
- **GitHub Codespaces.** 2 free vCPU hours daily per user; enough for Stage 1–4 development on a Chromebook.
- **Hugging Face Spaces (CPU).** A free place to host the MVP for a tiny user base while the paid VM is sized.

18.6 Production: What Lives on the Single VM

- Flask app behind Caddy / Nginx + TLS.
- SQLite (file) + HDF5 (file).
- `outputs/` directory for GDSII and PNGs.
- A tiny systemd service for the Python app.

- Nightly cron: `tar + rclone` to an off-site bucket.

Not on the VM: a database server, a message broker, an object store, a logging stack, a Kubernetes control plane. All of those can arrive in later stages when there are paying users to support them.

18.7 Scaling Beyond the MVP

The architecture scales smoothly *by adding tiers*, not by rewriting. A plausible path:

1. Replace SQLite with Postgres (same schema).
2. Replace the thread-pool job manager with RQ or Celery backed by Redis.
3. Move `outputs/` to S3-compatible storage.
4. Add workers that pull from the queue; the Flask app stays thin.
5. Optional GPU workers for Stage 6 jobs.

Nothing in the scientific core changes; the composition root simply swaps implementations of the `ArtifactStore` and job manager interfaces.

Chapter 19

Startup / Product Strategy

19.1 A Realistic Trajectory

The path from research project to small company is not a single leap. Treat each step as a product in its own right – useful even if the next step never ships.

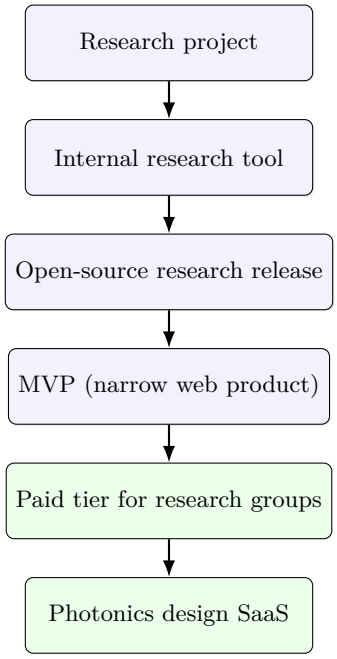


Figure 19.1: Suggested product trajectory.

19.2 Who Pays First?

- **Academic groups** who want a reproducible layout pipeline without the cost or lock-in of Lumerical / Comsol. Price point: a group licence paid once, institutional invoice.
- **Independent researchers** working with low-cost foundries (NanoLiten, LioniX, AMO) who need validated GDSII fast.
- **Educators** looking for teaching infrastructure; an institutional licence that supports a lab

class.

- **Early-stage photonics startups** who need a quick prototype layout without buying a full CAD seat.

Early customers are *not* enterprise semiconductor shops; those buyers have incumbents and buying cycles that eat small teams. Start where the budget is small enough that purchasing is a few emails and the problem is narrow enough that the MVP solves it today.

19.3 Narrow Positioning

A one-sentence positioning statement:

> *MetaCode is a reproducible, open metalens-design pipeline: RCWA-validated unit cells, a focusing phase library, and a fabrication-ready GDSII, from one form, on a laptop budget.*

Everything the project does should be defensible from that sentence. Features that do not sharpen the sentence are candidates for the backlog or the discard pile.

19.4 Open Core vs. Commercial

- **Open:** the scientific core, the templates, the analysis, the GDSII writer, the dependency manifest, the validation suite, the CLI.
- **Commercial (later):** the hosted MVP, access to precomputed sweep libraries over the full material set, team workspaces, retention of historical jobs, dedicated cloud GPUs for Stage 6 inverse design, support hours.

The commercial tier should never withhold scientific correctness; anything that could affect numbers lives in the open core.

19.5 Early Customer Experiments

- **Experiment 1: "twenty researchers".** Ship the MVP, email twenty researchers whose papers MetaCode cites, offer free access in exchange for a short structured interview. Measure: completed designs, time-to-first-GDSII, qualitative barriers.
- **Experiment 2: "one teaching course".** Offer a free institutional licence to one photonics course; measure: student completion of a design assignment.
- **Experiment 3: "one fabricated device".** Pay a foundry run for one lens designed through the MVP; publish the measured vs. simulated comparison. This is the credibility milestone.

19.6 Pricing Hypotheses

Low-friction, defensible:

- **Free tier.** Three jobs per week, three templates, standard materials.
- **Group tier.** $30–$60/month per group, 100 jobs, full material set, saved job history.
- **Institutional tier.** $500–$2000/year, course licence, offline mode (dockerised).

Numbers are hypotheses, not forecasts. The right numbers come from experiments 1–3.

19.7 Risks Specific to This Market

- **Grant-cycle buying.** Academic customers buy on grant calendars; shorten the sales loop by making the free tier instantly useful.
- **Validation anxiety.** Early buyers will ask "has anyone fabricated a lens using this?" Experiment 3 exists to answer that question.
- **Open-source commoditisation.** Someone else may publish a similar pipeline; the moat is the validation suite and the fabrication case study, not the code.

Chapter 20

Timeline, Milestones, and Risks

20.1 Milestone Table

Table 20.1 is the full milestone list. "Work" cells name the single thing that moves the stage from *pending* to *done*. Rough durations assume a single developer; a small team can parallelise independent rows.

Table 20.1: MetaCode milestone table.

M	Stage	Work	Papers	Deliverable	Est.
M1	1	Foundations notebooks + unit audit	[3, 7]	Internal docs + smoke tests	2 w
M2	2	Dispersive materials in MetaboxBackend; S4 adapter stub	[17, 16]	Cross-plots metabox vs. S4	3 w
M3	3	SQLite+HDF5 `ArtifactStore`; sweep cache	–	Persistent store + CLI ingest	2 w
M4	4	`design_metalens(...)` service + GDSII	[7, 11]	GDSII for each template	3 w
M5	5	Meep backend (unit-cell path)	[20]	Cross-check report	4 w
M6	6a	Harden GradientOptimizer; seedable TF	[8]	Reproducible adjoint run	2 w
M7	6b	Optuna outer loop	[1]	Beats random search baseline	3 w
M8	6c	Trained surrogate; wire into SurrogateOptimizer	[9]	Surrogate RMSE $<$ target	4 w
M9	7	Thread-pool job manager; Report dataclass	–	Async pipeline	2 w
M10	8	SQLite+HDF5 store wired into tool registry; Flask API	–	Transport-neutral tool surface	3 w
M11	10	Validation suite (level-0, level-1)	[11]	`pytest` green on main	2 w
M12	11a	MVP Flask skeleton + form + job queue	–	Deployable Flask app	3 w
M13	11b	Three outside users + fabricated-device case study	–	First measured-vs-simulated comparison	8 w
M14	9	[OPTIONAL] Chat adapter + numeric-faithfulness check	[2]	Opt-in chat form	3 w

20.2 Dependencies Between Milestones

20.3 Phase / Difficulty / Risk / Exit

20.4 Risk Register

Simulation accuracy. Mitigation: Stage 5 cross-check; Stage 10 validation suite.

Material models. Mitigation: CSV-based $n(\lambda)$ in the metabox backend (Stage 2 task).

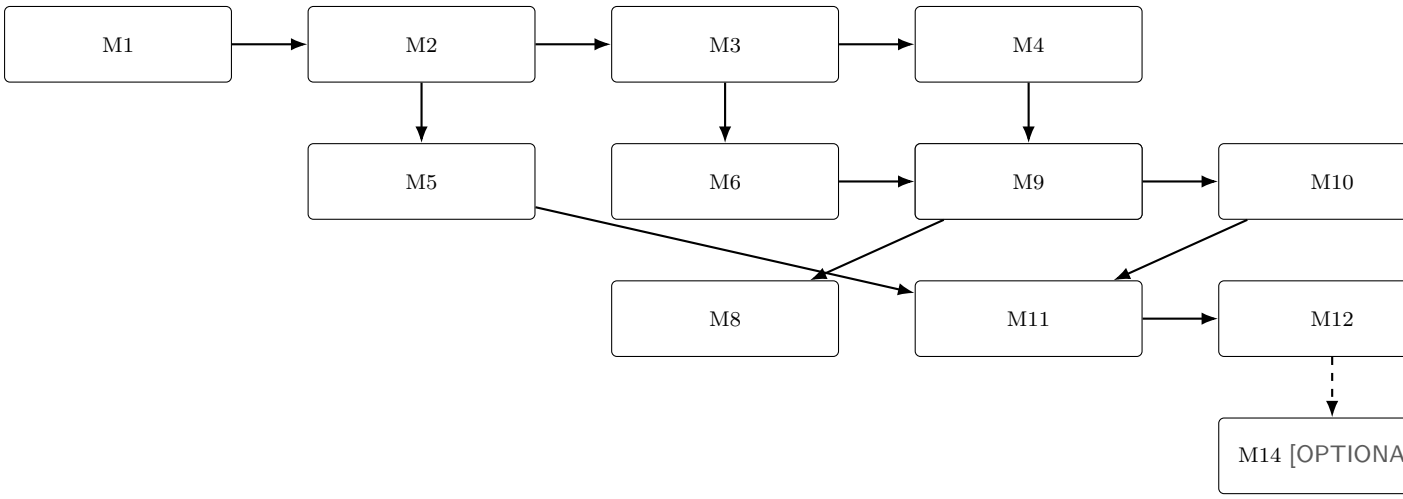


Figure 20.1: Milestone dependency DAG. M14 (optional chat) is dashed; it is not a prerequisite for any other milestone.

Table 20.2: Phases, difficulty, risk, exit criteria.

Phase	**Dependencies**	**Diff.**	**Risk**	**Exit**
1 Foundations	–	Low	Low	Smoke tests pass.
2 Unit-cell engine	1	Med	Low	metabox vs. S4 agree per template.
3 DB	2	Low	Low	Cache hit for identical input.
4 Metalens design	3	Med	Low	GDSII per template loads in KLayout.
5 FDTD validation	2	High	Med	Meep agrees with metabox within tol.
6 Inverse design	2, 3	High	Med	Beats random-search baseline.
7 Automation	4	Low	Low	Replay reproduces metrics.
8 Transport	3, 7	Low	Low	Integration test passes per transport.
9 LLM (optional)	8	Med	Med	Opt-in form routes through adapter.
10 Validation suite	2, 5	Med	Low	Level-0/1 green on main.
11 MVP	7, 8, 10	Med	Med	Three outside users ship a design.

Compute cost. Mitigation: precompute, cache, approximate (Chapter 18).

Numerical convergence. Mitigation: grid-refinement study baked into Stage 5.

Optimisation complexity. Mitigation: Optuna budget caps + surrogate shortcut (Stage 6).

Fabrication constraints. Mitigation: tolerance scan (Stage 4 task); design-rule check in a later stage.

Database quality. Mitigation: provenance fields in every row; validation suite diffs stored sweeps against live reruns.

User adoption. Mitigation: narrow positioning (Chapter 19); Experiment 3 fabricated device.

Cloud costs. Mitigation: single VM throughout MVP; precomputed caches.

LLM hallucination / tool misuse. Mitigation: numeric- faithfulness check; LLM optional and off by default.

Model-provider drift (Groq or any other API). Mitigation: provider-neutral adapter; local open-weights fallback.

Chapter 21

Conclusion and Recommended Execution Order

21.1 Summary

MetaCode is small enough that a plausible path to a usable product is visible, and ambitious enough that the path is worth walking. The scientific core is already present; the roadmap's only task is composition – backends, storage, orchestration, transport – around work that already exists. The product plan deliberately avoids expensive bets. The MVP is a Flask app on a $5/month VM; everything else is deferred until that app has at least three outside users who finish a design end-to-end.

21.2 The One-Page Execution Order

A single ordered checklist, intended to be printed and pinned to a wall. Items marked [OPTIONAL] may be skipped without blocking the rest.

1. [NOW] Stage 1: audit units, write foundations notebooks, replace UTF-16 `requirements.txt` with `pyproject.toml`.
2. [NOW] Stage 2: make `MetaboxBackend` use CSV-based $n(\lambda)$; implement `S4Backend`; write `compare_backends` notebook.
3. [NEXT] Stage 3: build the SQLite+HDF5 `ArtifactStore`; cache sweeps; add CLI ingest.
4. [NEXT] Stage 4: service `design_metalens(params, sweep_id, n_levels)` driving the existing phase/quantise/GDS pipeline.
5. [NEXT] Stage 5: Meep adapter for `simulate_unit_cell`; one cross-check figure per shipped material.
6. [NEXT] Stage 7: thread-pool job manager; async pipeline; `Report` dataclass.
7. [NEXT] Stage 10: validation suite (level-0, level-1); CI on push.
8. [NEXT] Stage 6: harden gradient optimiser; Optuna outer loop; keep behind an opt-in flag.

9. [MVP] Stage 8/11: SQLite+HDF5 store wired into `build_tool_registry`; Flask MVP with form, job queue, downloads, history; deploy to a $5 VM.
10. [MVP] Experiment 1: twenty researchers, free access, structured interviews.
11. [MVP] Experiment 3: pay for one foundry run through the MVP and publish the measured-vs-simulated figure.
12. [OPTIONAL] Stage 9: provider-neutral chat adapter; numeric-faithfulness check; opt-in only.
13. [LATER] Stage 6c: train a surrogate; wire into `SurrogateOptimizer`.
14. [LATER] Scale: Postgres → queue → workers → GPU workers; commercial tiers.

21.3 Final Words

The hardest discipline this roadmap asks for is *resisting scope creep*. Every stage names what it does not do. The MVP list of non-features is as important as its feature list. The LLM stage is explicitly optional. The validation suite is non-negotiable before anything is shipped to outside users. If the roadmap is executed in order and in that discipline, MetaCode can become a defensible, open photonics tool with a credible narrow product – without a cloud bill, without an enterprise sales motion, and without pretending to be a commercial CAD replacement.

The next unit of work is Stage 1. It will take a fortnight. That is the right first step.

Bibliography

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proc. 25th ACM SIGKDD*, 2019.

[2] Anthropic. Model context protocol: Specification. https://modelcontextprotocol.io/, 2024.

[3] Max Born and Emil Wolf. *Principles of Optics*. Cambridge University Press, 7th edition, 1999.

[4] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. http://github.com/google/jax, 2018.

[5] Hou-Tong Chen, Antoinette J. Taylor, and Nanfang Yu. A review of metasurfaces: Physics and applications. *Reports on Progress in Physics*, 79(7):076401, 2016.

[6] Wei Ting Chen, Alexander Y. Zhu, and Federico Capasso. Flat optics with dispersion-engineered metasurfaces. *Nature Reviews Materials*, 5:604–620, 2020.

[7] Joseph W. Goodman. *Introduction to Fourier Optics*. W. H. Freeman, 4th edition, 2017.

[8] Tyler W. Hughes, Ian A. D. Williamson, Momchil Minkov, and Shanhui Fan. Forward-mode differentiation of Maxwell's equations. *ACS Photonics*, 6(11):3010–3016, 2019.

[9] Jiaqi Jiang, Mingkun Chen, and Jonathan A. Fan. Deep neural networks for the evaluation and design of photonic devices. *Nature Reviews Materials*, 6:679–700, 2021.

[10] Joaquin Matres and contributors. GDSFactory: Open-source python framework for chip design automation. https://gdsfactory.github.io/gdsfactory/, 2024.

[11] Mohammadreza Khorasaninejad, Wei Ting Chen, Robert C. Devlin, Jaewon Oh, Alexander Y. Zhu, and Federico Capasso. Metalenses at visible wavelengths: Diffraction-limited focusing and subwavelength resolution imaging. *Science*, 352(6290):1190–1194, 2016.

[12] Matthias Köfferlein and contributors. KLayout: Layout viewer and editor. https://www.klayout.de/, 2024.

[13] Philippe Lalanne and G. Michael Morris. Highly improved convergence of the coupled-wave method for tm polarization. *Journal of the Optical Society of America A*, 13(4):779–784, 1996.

[14] Thorsten Liebig, Andreas Rennings, Sebastian Held, and Daniel Erni. openems – a free and open source equivalent-circuit (ec) FDTD simulation platform. *International Journal of Numerical Modelling*, 2013.

[15] Dianmin Lin, Pengyu Fan, Erez Hasman, and Mark L. Brongersma. Dielectric gradient metasurface optical elements. *Science*, 345(6194):298–302, 2014.

[16] Victor Liu and Shanhui Fan. S4: A free electromagnetic solver for layered periodic structures. *Computer Physics Communications*, 183(10):2233–2244, 2012.

[17] M. G. Moharam, Eric B. Grann, Drew A. Pommet, and T. K. Gaylord. Formulation for stable and efficient implementation of the rigorous coupled-wave analysis of binary gratings. *Journal of the Optical Society of America A*, 12(5):1068–1076, 1995.

[18] Sean Molesky, Zin Lin, Alexander Y. Piggott, Weiliang Jin, Jelena Vucković, and Alejandro W. Rodriguez. Inverse design in nanophotonics. *Nature Photonics*, 12(11):659–670, 2018.

[19] Robert J. Noll. Zernike polynomials and atmospheric turbulence. *Journal of the Optical Society of America*, 66(3):207–211, 1976.

[20] Ardavan F. Oskooi, David Roundy, Mihai Ibanescu, Peter Bermel, J. D. Joannopoulos, and Steven G. Johnson. MEEP: A flexible free-software package for electromagnetic simulations by the FDTD method. *Computer Physics Communications*, 181(3):687–702, 2010.

[21] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

[22] Raphaël Pestourie, Carlos Pérez-Arancibia, Zin Lin, Wonseok Shin, Federico Capasso, and Steven G. Johnson. Inverse design of large-area metasurfaces. *Optics Express*, 26(26):33732–33747, 2018.

[23] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[24] Nanfang Yu, Patrice Genevet, Mikhail A. Kats, Francesco Aieta, Jean-Philippe Tetienne, Federico Capasso, and Zeno Gaburro. Light propagation with phase discontinuities: Generalized laws of reflection and refraction. *Science*, 334(6054):333–337, 2011.